

MathModule QA Documentation

Overview

The MathModule is a comprehensive JavaScript class for handling mathematical formulas in a rich text editor environment. It supports both inline and display formulas with various client-specific configurations.

Element Order and Structure

1. Formula Element Hierarchy

Inline Formula Structure

```
span.inline-formula (root)
├─ span.TEX (contains LaTeX code)
└─ span.inline-graphic (contains SVG/image)
```

Display Formula Structure

```
div.disp-formula (root)
├─ span.TEX (contains LaTeX code)
├─ span.graphic (contains SVG/image)
└─ span.label (equation number - if sequential)
```

Client-Specific Variations

PLOS Client:

```
div.disp-formula
├─ span.alternatives
│   ├── span.TEX
│   └─ span.graphic
└─ span.label (OUTSIDE of alternatives)
```

OUP/LWW Clients:

```
div.disp-formula
├─ span.TEX
├─ span.graphic
└─ span.label (OUTSIDE of graphic)
```

2. Element Processing Order

1. Initialization: `showLoop()` → sets type and sequence flags
2. Element Creation: `getNewItem()` → creates DOM structure

- 3. Content Update: `checkandUpdate_svgView()` → updates existing elements
- 4. Label Management: `ReOrderingEqLabel()` → renumbers equations
- 5. Insertion: `insertMath()` → places element in document

generateId Function Analysis

Function Signature

```
generateId(e = "inline")
```

ID Generation Logic Table

Client	ID Format	PDF ID Format	Counter Logic
OUP	<code>IN{4-digit}</code> (inline) <code>M{number}</code> (display)	<code>{projectname}_{id}.pdf</code>	Scans existing formulas for highest number
PLOS	<code>{journal}.e{3-digit}</code>	<code>{id}.pdf</code>	Uses 3-digit padded counter
LWW	<code>{uuid}</code>	<code>{projectname}_M{4-digit}.pdf</code>	Uses UUID with 4-digit counter
Default	<code>{uuid}</code>	<code>default_{uuid}.pdf</code>	Fallback UUID generation

Real-World Examples

Client	Type	Output.id	Output.pdfId
OUP	inline	<code>IN0001</code>	<code>CLEANE_zkaf028_IN0001.pdf</code>
OUP	display	<code>M1</code>	<code>CLEANE_zkaf028_M1.pdf</code>
PLOS	inline	<code>pwat.0000299.e001</code>	<code>pwat.0000299.e001.pdf</code>
PLOS	display	<code>pwat.0000299.e002</code>	<code>pwat.0000299.e002.pdf</code>
LWW	any	<code>random</code> (via <code>s4()</code>)	<code>MD-D-25-05300_M0001.pdf</code>

Return Value Examples

Client	Type	Return.id	Return.pdfId
OUP	inline	<code>IN0001</code>	<code>CLEANE_zkaf028_IN0001.pdf</code>
OUP	display	<code>M1</code>	<code>CLEANE_zkaf028_M1.pdf</code>
PLOS	inline	<code>pwat.0000299.e001</code>	<code>pwat.0000299.e001.pdf</code>
LWW	display	<code>a8f93d21</code>	<code>MD-D-25-05300_M0001.pdf</code>
Unknown	any	<code>random-uuid</code>	<code>default_random-uuid.pdf</code>

ID Generation Test Cases

Test Case	Input	Expected Output (OUP)	Expected Output (PLOS)	Expected Output (LWW)
First inline formula	"inline"	{id: "IN0001", pdfId: "PROJECT_IN0001.pdf"}	{id: "journal.e001", pdfId: "journal.e001.pdf"}	{id: "uuid", pdfId: "PROJECT_M0001.pdf"}
First display formula	"display"	{id: "M1", pdfId: "PROJECT_M1.pdf"}	{id: "journal.e001", pdfId: "journal.e001.pdf"}	{id: "uuid", pdfId: "PROJECT_M0001.pdf"}
After existing formulas	N/A	Increments from highest found	Increments from highest found	Increments from highest found

Counter Logic Details

```
// Counter scanning logic
s.querySelectorAll(".inline-formula, .disp-formula").forEach(e => {
  let t = e.getAttribute("id");
  let a = t?.match(/(\d{3,4})$/); // Extract 3-4 digit number
  if (a) {
    let r = parseInt(a[1], 10);
    if (!isNaN(r) && r > 1) {
      l = r; // Update maximum counter
    }
  }
});
```

QA Test Scenarios

1. Element Order Validation

Test Case: Inline Formula Creation

```
// Setup
mathModule.type = "inline";
mathModule.seq = false;

// Expected DOM structure
<span class="inline-formula" id="generated-id">
  <span class="TEX">$$latex-code$$</span>
  <span class="inline-graphic">
    <svg>...</svg>
  </span>
</span>

// Client PLOS in XML
<inline-formula id="pone.0328148.e002">
  <alternatives>
    <graphic id="pone.0328148.e002g" position="anchor" mimetype="image" xlink:href="pone.0328148.e002.pdf" x:
  </alternatives>
</inline-formula>
```

Test Case: PLOS Display Formula with Sequence

```
// Setup
mathModule.type = "display";
mathModule.seq = true;
// Client: PLOS

// Expected DOM structure
<div class="disp-formula" id="generated-id">
  <span class="alternatives">
    <span class="TEX">$\latex-code$</span>
    <span class="graphic">
      <svg>...</svg>
    </span>
  </span>
  <span class="label">(1)</span>
</div>

// Excepted in XML Structure
<disp-formula id="pone.0328148.e001"><?PageID 8?>
  <alternatives>
    <graphic id="pone.0328148.e001g" position="anchor" mimetype="image" xlink:href="pone.0328148.e001.pdf">
  </alternatives>
  <label>(1)</label>
</disp-formula>
```

2. ID Generation Validation

Test Case: OUP Client ID Generation

```
// Test data
SHARED_KEY = {
  client: "OUP",
  projectname: "TEST_PROJECT",
  fileId: "file123",
  titleinfo: { cover: "cover123" },
  identifier: "path/to/OUN_ARTICLE"
};

// Expected results
generateId("inline") → {id: "IN0001", pdfId: "TEST_PROJECT_IN0001.pdf"}
generateId("display") → {id: "M1", pdfId: "TEST_PROJECT_M1.pdf"}
```

Test Case: PLOS Client ID Generation

```
// Test data
SHARED_KEY = {
  client: "PLOS",
  identifier: "journal.pone.0123456"
};

// Expected results
generateId() → {id: "pone.0123456.e001", pdfId: "pone.0123456.e001.pdf"}
```

3. Label Reordering Validation

Test Case: Equation Renumbering

```
// Initial state: equations with labels (1), (3), (2)
// After ReOrderingEqLabel()
// Expected: equations renumbered to (1), (2), (3)

// Validation points:
1. eqOrder object contains correct mapping
2. DOM labels updated to sequential numbers
3. Citation links updated to match new numbers
4. Track attributes preserved for change tracking
```

Test Case: OUP/LWW Display Formula with Sequence

```
// Setup
mathModule.type = "display";
mathModule.seq = true;
// Client: OUP or LWW

// Expected DOM structure
<div class="disp-formula" id="generated-id">
  <span class="TEX">$\latex-code$</span>
  <span class="graphic">
    <svg>...</svg>
  </span>
  <span class="label">(1)</span>
</div>
```

Error Handling Test Cases

1. Fallback ID Generation

```
// Test when all client-specific logic fails
// Expected: UUID-based fallback
{id: "uuid-string", pdfId: "fallback_uuid-string.pdf"}
```

2. Missing Dependencies

```
// Test when GlobalEditor is undefined
// Expected: Graceful error handling with console.warn
```

3. Invalid Element Types

```
// Test with non-existent element type
mathModule.type = "invalid";
// Expected: Default to "inline" type
```

Performance Considerations

1. DOM Query Optimization

- `querySelectorAll` calls should be minimized
- Cache frequently accessed elements
- Use efficient selectors

2. Memory Management

- Clear references after use
- Avoid memory leaks in event handlers
- Proper cleanup of temporary objects

Browser Compatibility

Supported Features

- ES6 classes and arrow functions
- `querySelector/querySelectorAll`
- Template literals
- `Object.assign` and `Object.entries`

Fallback Requirements

- Polyfills for older browsers
- Alternative implementations for unsupported features

Integration Points

1. CKEditor Integration

- Command registration: `ckeditor_wiris_openFormulaEditor`
- Selection handling: `IMPACT_SELECTION`
- Document manipulation: `GlobalEditor.document`

2. External Dependencies

- `WirisPlugin` for LaTeX processing
- `trackManager` for change tracking
- `commonMethods` for utility functions

Validation Checklist

- ☐ Element structure matches client requirements
- ☐ ID generation follows client-specific patterns
- ☐ Counter logic correctly identifies highest existing number
- ☐ Label renumbering maintains sequential order
- ☐ Error handling prevents crashes
- ☐ Performance meets requirements
- ☐ Browser compatibility verified

-  browser compatibility verified

- ☐ Integration points function correctly